

CyberGuard Pro - Software Requirements Specification (SRS) Document

Table of Contents

1. Introduction
 2. Overall Description
 3. System Features
 4. External Interface Requirements
 5. System Quality Attributes
 6. Technical Architecture
 7. Implementation Plan
 8. Testing Strategy
 9. Risk Analysis
 10. Appendices
-

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive description of the CyberGuard Pro desktop cybersecurity application. It outlines the system's functionality, performance requirements, design constraints, and implementation specifications for a 6-member development team.

1.2 Scope

CyberGuard Pro is a comprehensive desktop cybersecurity application designed to protect users' computers and manage their digital security. The system integrates multiple security modules including real-time threat monitoring, malware detection, password management, and advanced NLP-based threat analysis.

Key Capabilities:

- Real-time system monitoring and threat detection
- File security scanning with malware detection
- Secure password management with encryption
- Network security monitoring
- NLP-powered cyber threat analysis
- Automated threat prevention system

- Comprehensive security reporting

1.3 Definitions, Acronyms, and Abbreviations

- **API:** Application Programming Interface
- **GUI:** Graphical User Interface
- **ML:** Machine Learning
- **NLP:** Natural Language Processing
- **SRS:** Software Requirements Specification
- **UI/UX:** User Interface/User Experience
- **SQLite:** Embedded relational database
- **PyQt/Tkinter:** Python GUI frameworks

1.4 References

- IEEE Std 830-1998 - IEEE Recommended Practice for Software Requirements Specifications
 - Python Security Best Practices
 - OWASP Top 10 Security Risks
 - NIST Cybersecurity Framework
-

2. Overall Description

2.1 Product Perspective

CyberGuard Pro operates as a standalone desktop application that integrates with the host operating system to provide comprehensive cybersecurity protection. The system consists of interconnected modules that work together to monitor, detect, and prevent security threats.

System Context Diagram:



2.2 Product Functions

The system provides the following major functions:

1. **Security Monitoring**

- Real-time system resource monitoring
- Process and network activity tracking
- Suspicious activity detection

2. **File Security**

- Malware signature detection
- File integrity checking
- Quarantine management

3. **Password Management**

- Secure password storage and retrieval
- Password strength analysis
- Automatic password generation

4. **Threat Detection**

- NLP-based threat analysis
- Pattern recognition for phishing attempts
- Social engineering detection

5. **Automated Response**

- Threat blocking and quarantine
- User notifications and alerts
- Security report generation

2.3 **User Characteristics**

Primary Users:

- Individual computer users seeking enhanced security
- Small business owners protecting company data
- Students and professionals handling sensitive information

Technical Expertise:

- Basic to intermediate computer literacy
- No cybersecurity expertise required
- Ability to install and configure desktop software

2.4 **Constraints**

Development Constraints:

- 12-week development timeline
- 6-person development team
- Python-based implementation
- Cross-platform compatibility (Windows, macOS, Linux)

Technical Constraints:

- Desktop application (no web-based components)
- Local data storage using SQLite
- Memory usage optimization required
- Real-time processing capabilities

Regulatory Constraints:

- Data privacy compliance
 - Local data encryption requirements
 - No external data transmission without user consent
-

3. System Features

3.1 Real-Time System Monitoring

Description: Continuously monitors system resources, processes, and network activity to detect potential security threats.

Functional Requirements:

- FR-SM-01: System shall monitor CPU, memory, disk, and network usage
- FR-SM-02: System shall detect processes consuming abnormal resources
- FR-SM-03: System shall track network connections and data transfer
- FR-SM-04: System shall identify suspicious process behavior
- FR-SM-05: System shall provide real-time dashboard updates

Priority: High **Stimulus/Response:** System resource changes trigger monitoring updates **Acceptance Criteria:**

- Monitor updates every 5 seconds
- Detect processes using >80% CPU or >50% memory
- Track all network connections with protocol identification

3.2 File Security Scanner

Description: Scans files and directories for malware, viruses, and other security threats using signature-based detection.

Functional Requirements:

- FR-FS-01: System shall scan individual files on demand
- FR-FS-02: System shall perform full directory scanning
- FR-FS-03: System shall compare files against malware signature database
- FR-FS-04: System shall calculate and verify file hashes
- FR-FS-05: System shall quarantine detected threats

Priority: High **Stimulus/Response:** File access or user scan request triggers security analysis **Acceptance Criteria:**

- Scan speed: >1000 files per minute
- Detection accuracy: >95% for known malware signatures
- Zero false positives for system files

3.3 Password Manager

Description: Securely stores, manages, and generates passwords with strong encryption.

Functional Requirements:

- FR-PM-01: System shall encrypt passwords using AES-256 encryption
- FR-PM-02: System shall generate strong passwords with customizable criteria
- FR-PM-03: System shall assess password strength and provide recommendations
- FR-PM-04: System shall organize passwords by website/application
- FR-PM-05: System shall provide secure password retrieval

Priority: Medium **Stimulus/Response:** User password management requests trigger secure operations **Acceptance Criteria:**

- Master password required for access
- Password generation includes uppercase, lowercase, numbers, symbols
- Encryption/decryption time <100ms per operation

3.4 NLP Threat Detection

Description: Uses Natural Language Processing and Machine Learning to analyze text content for cyber threats, phishing attempts, and social engineering.

Functional Requirements:

- FR-NLP-01: System shall analyze text content using NLP algorithms
- FR-NLP-02: System shall detect phishing patterns in emails and messages
- FR-NLP-03: System shall identify social engineering attempts
- FR-NLP-04: System shall classify threat levels (safe, low, medium, high, critical)
- FR-NLP-05: System shall provide confidence scores for threat assessments

Priority: High **Stimulus/Response:** Text analysis requests trigger NLP processing **Acceptance Criteria:**

- Analysis time <5 seconds for 1000-word text
- Threat classification accuracy >85%
- Support for multiple threat pattern types

3.5 Automated Threat Prevention

Description: Automatically responds to detected threats by blocking, quarantining, or alerting users.

Functional Requirements:

- FR-TP-01: System shall automatically block high-risk domains
- FR-TP-02: System shall quarantine malicious files
- FR-TP-03: System shall generate user alerts for detected threats
- FR-TP-04: System shall log all threat prevention actions
- FR-TP-05: System shall allow user override of prevention actions

Priority: High **Stimulus/Response:** Threat detection triggers automated prevention measures

Acceptance Criteria:

- Response time <1 second for threat blocking
- Maintain audit log of all prevention actions
- User notification within 5 seconds of threat detection

3.6 Security Dashboard and Reporting

Description: Provides comprehensive security status overview and generates detailed security reports.

Functional Requirements:

- FR-SD-01: System shall display real-time security status dashboard
- FR-SD-02: System shall generate periodic security reports
- FR-SD-03: System shall track security metrics and trends
- FR-SD-04: System shall export reports in multiple formats (PDF, CSV)

- FR-SD-05: System shall maintain scan history and threat logs

Priority: Medium **Stimulus/Response:** User requests for security information trigger report generation

Acceptance Criteria:

- Dashboard updates in real-time
 - Report generation <30 seconds
 - Export functionality for all major formats
-

4. External Interface Requirements

4.1 User Interface Requirements

GUI Requirements:

- Modern, intuitive desktop interface using CustomTkinter or PyQt5
- Responsive design supporting 1024x768 minimum resolution
- Consistent color scheme and typography
- Accessibility features for users with disabilities
- Multi-language support (English primary)

Interface Components:

- Main dashboard with security status overview
- Navigation sidebar with module access
- Real-time monitoring graphs and charts
- Alert and notification system
- Settings and configuration panels

4.2 Hardware Interfaces

Minimum System Requirements:

- Operating System: Windows 10+, macOS 10.14+, Ubuntu 18.04+
- Processor: Intel i3 2.0GHz or AMD equivalent
- Memory: 4GB RAM minimum, 8GB recommended
- Storage: 2GB available disk space
- Network: Internet connection for updates and threat database

4.3 Software Interfaces

Database Interface:

- SQLite 3.x for local data storage
- Database schema for user settings, scan results, password storage
- Encrypted data storage for sensitive information

Operating System Interface:

- File system access for scanning operations
- Network monitoring capabilities
- Process and system resource monitoring
- Registry/system configuration access (Windows)

External Libraries:

- Python 3.8+ runtime environment
- Cryptography library for encryption operations
- NLTK/spaCy for natural language processing
- psutil for system monitoring
- Requests for network operations

4.4 Communication Interfaces

Network Communication:

- HTTPS connections for security updates
 - Local network monitoring for threat detection
 - No unauthorized external data transmission
 - Optional telemetry with user consent
-

5. System Quality Attributes

5.1 Performance Requirements

Response Time:

- GUI interactions: <200ms response time
- File scanning: >1000 files per minute
- System monitoring updates: 5-second intervals
- Database operations: <100ms for basic queries

Throughput:

- Concurrent file scanning capability
- Real-time monitoring without system impact
- Efficient memory usage (<500MB baseline)

Resource Utilization:

- CPU usage <10% during idle monitoring
- Memory footprint optimization
- Minimal disk I/O impact during scanning

5.2 Security Requirements

Data Protection:

- AES-256 encryption for stored passwords
- Secure key derivation functions (PBKDF2/scrypt)
- Local data storage only (no cloud synchronization)
- Encrypted database storage for sensitive data

Access Control:

- Master password authentication
- Session timeout management
- Secure memory handling for sensitive data
- Protection against memory dumps

5.3 Reliability Requirements

Availability:

- 99.9% uptime for monitoring services
- Automatic recovery from component failures
- Graceful degradation of non-critical features
- Data integrity protection during system crashes

Error Handling:

- Comprehensive exception handling
- Logging system for debugging
- User-friendly error messages
- Automatic error reporting (with user consent)

5.4 Usability Requirements

Ease of Use:

- Intuitive interface requiring minimal training
- Clear navigation and information hierarchy
- Contextual help and documentation
- Consistent user experience across modules

Accessibility:

- Keyboard navigation support
- High contrast mode availability
- Scalable font sizes
- Screen reader compatibility

5.5 Maintainability Requirements

Code Quality:

- Modular architecture with clear separation of concerns
- Comprehensive code documentation
- Unit test coverage >80%
- Consistent coding standards and style

Extensibility:

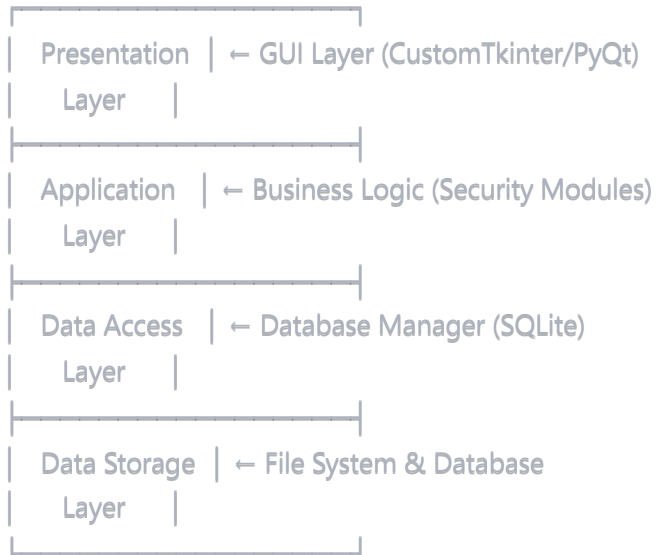
- Plugin architecture for additional security modules
 - Configurable threat detection rules
 - API for third-party integrations
 - Update mechanism for security definitions
-

6. Technical Architecture

6.1 System Architecture Overview

Architecture Pattern: Model-View-Controller (MVC)

- **Model:** Data layer (SQLite database, file system)
- **View:** GUI layer (CustomTkinter/PyQt interface)
- **Controller:** Business logic (security modules, threat detection)



6.2 Component Architecture

Core Components:

1. **GUI Manager** - User interface coordination
2. **Security Scanner** - File and system scanning
3. **System Monitor** - Real-time resource monitoring
4. **Password Manager** - Secure credential storage
5. **NLP Threat Detector** - Intelligent threat analysis
6. **Database Manager** - Data persistence layer
7. **Threat Prevention System** - Automated response mechanisms

6.3 Data Architecture

Database Schema:

sql

```

-- Users table
CREATE TABLE users (
    id INTEGER PRIMARY KEY,
    username TEXT UNIQUE NOT NULL,
    password_hash TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Scan history table
CREATE TABLE scan_history (
    id INTEGER PRIMARY KEY,
    scan_type TEXT NOT NULL,
    scan_path TEXT NOT NULL,
    threats_found INTEGER DEFAULT 0,
    scan_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    results TEXT
);

-- Password storage table
CREATE TABLE stored_passwords (
    id INTEGER PRIMARY KEY,
    website TEXT NOT NULL,
    username TEXT NOT NULL,
    encrypted_password TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Threat logs table
CREATE TABLE threat_logs (
    id INTEGER PRIMARY KEY,
    threat_type TEXT NOT NULL,
    threat_level TEXT NOT NULL,
    source TEXT,
    details TEXT,
    action_taken TEXT,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

6.4 Technology Stack

Programming Language: Python 3.8+

GUI Framework: CustomTkinter (Primary) / PyQt5 (Alternative)

Database: SQLite 3.x

Core Libraries:

- `psutil` - System monitoring
- `cryptography` - Encryption operations
- `watchdog` - File system monitoring
- `sqlite3` - Database operations
- `hashlib` - Hash functions
- `threading` - Concurrent processing

NLP/ML Libraries:

- `transformers` - Hugging Face transformers
- `torch` - PyTorch for ML models
- `scikit-learn` - Machine learning algorithms
- `nltk` - Natural language processing
- `spacy` - Advanced NLP processing
- `pandas` - Data manipulation
- `numpy` - Numerical computing

Security Libraries:

- `yara-python` - Advanced malware detection (optional)
 - `scapy` - Network packet analysis (optional)
-

7. Implementation Plan

7.1 Development Methodology

Approach: Agile development with 2-week sprints **Team Structure:** 6-member cross-functional team

Timeline: 12-week development cycle

7.2 Team Roles and Responsibilities

Person 1 - GUI Developer (Frontend)

- Design and implement user interface
- Create responsive and intuitive layouts
- Handle user interactions and events
- Integrate frontend with backend services

Person 2 - System Security Developer

- Implement file scanning functionality

- Develop malware detection algorithms
- Create real-time protection systems
- Handle quarantine operations

Person 3 - Database Developer

- Design and implement database schema
- Create data access layer
- Handle data persistence and retrieval
- Manage database migrations and updates

Person 4 - System Monitor Developer

- Implement real-time system monitoring
- Create network activity tracking
- Develop performance metrics collection
- Handle suspicious process detection

Person 5 - Password Manager Developer

- Implement secure password storage
- Create password generation algorithms
- Handle encryption/decryption operations
- Develop password strength analysis

Person 6 - Testing & Integration Manager + NLP Developer

- Implement NLP threat detection system
- Create machine learning models
- Integrate all system components
- Conduct comprehensive testing
- Manage bug tracking and resolution

7.3 Development Phases

Phase 1: Foundation (Weeks 1-2)

- Project setup and environment configuration
- Team role assignment and tool setup
- Basic project structure creation
- Initial component development

Phase 2: Core Development (Weeks 3-6)

- Individual component implementation
- Basic functionality development
- Unit testing for each module
- Initial integration testing

Phase 3: Advanced Features (Weeks 7-10)

- NLP threat detection implementation
- Advanced security features
- User interface enhancement
- Performance optimization

Phase 4: Integration & Testing (Weeks 11-12)

- System integration testing
- Bug fixes and performance tuning
- User acceptance testing
- Documentation and deployment

7.4 Milestones and Deliverables

Week 2: Project setup complete, basic components initialized **Week 4:** Core functionality implemented, basic GUI operational **Week 6:** Individual modules tested, initial system integration **Week 8:** Advanced features implemented, NLP system operational **Week 10:** System optimization complete, comprehensive testing begun **Week 12:** Final testing complete, documentation finished, deployment ready

8. Testing Strategy

8.1 Testing Approach

Testing Types:

1. **Unit Testing** - Individual component validation
2. **Integration Testing** - Component interaction verification
3. **System Testing** - End-to-end functionality validation
4. **Performance Testing** - Load and stress testing
5. **Security Testing** - Vulnerability assessment
6. **Usability Testing** - User experience validation

8.2 Test Coverage Requirements

Minimum Coverage:

- Unit tests: >80% code coverage
- Integration tests: All module interfaces
- System tests: All major user workflows
- Security tests: All data handling operations

8.3 Testing Tools

Automated Testing:

- `unittest` - Python unit testing framework
- `pytest` - Advanced testing framework
- `mock` - Test mocking library
- `coverage.py` - Code coverage measurement

Manual Testing:

- User acceptance testing protocols
- Security vulnerability assessments
- Performance benchmarking
- Cross-platform compatibility testing

8.4 Test Data Management

Test Environments:

- Development environment with sample data
 - Staging environment with production-like data
 - Isolated testing environment for security tests
 - Performance testing with large datasets
-

9. Risk Analysis

9.1 Technical Risks

High Risk:

- **NLP Model Performance:** Complex machine learning implementation may impact system performance

- **Real-time Processing:** Continuous monitoring may consume excessive system resources
- **Cross-platform Compatibility:** GUI framework differences across operating systems

Mitigation Strategies:

- Implement efficient algorithms with performance monitoring
- Use optimized libraries and caching mechanisms
- Extensive testing on all target platforms

9.2 Security Risks

High Risk:

- **Password Storage:** Vulnerabilities in encryption implementation
- **Privilege Escalation:** System-level access requirements
- **False Positives:** Incorrect threat detection causing user disruption

Mitigation Strategies:

- Use proven cryptographic libraries
- Implement least privilege access principles
- Extensive testing with diverse threat scenarios

9.3 Project Risks

Medium Risk:

- **Timeline Constraints:** 12-week development schedule
- **Team Coordination:** 6-member team synchronization
- **Scope Creep:** Feature additions beyond initial requirements

Mitigation Strategies:

- Regular sprint reviews and timeline monitoring
- Daily standups and clear communication channels
- Strict scope management with change control process

10. Appendices

10.1 Installation Requirements

System Requirements:

Operating System: Windows 10+ / macOS 10.14+ / Ubuntu 18.04+
Python: 3.8 or higher
Memory: 4GB RAM minimum (8GB recommended)
Storage: 2GB available space
Network: Internet connection for updates

Installation Commands:

```
bash

# Core requirements
pip install customtkinter
pip install psutil
pip install cryptography
pip install watchdog
pip install sqlite3
pip install requests

# NLP and ML libraries
pip install transformers
pip install torch
pip install scikit-learn
pip install nltk
pip install spacy
pip install pandas
pip install numpy
pip install joblib

# Download spaCy model
python -m spacy download en_core_web_sm

# Package for distribution
pip install pyinstaller
```

10.2 Configuration Guidelines

Database Configuration:

- SQLite database file: `cyberguard.db`
- Automatic schema creation on first run
- Regular backup recommendations

Security Configuration:

- Strong master password requirements

- Encryption key derivation settings
- Quarantine directory permissions

10.3 Performance Benchmarks

Target Performance Metrics:

- Application startup time: <5 seconds
- File scanning rate: >1000 files/minute
- Memory usage: <500MB baseline
- CPU usage during idle: <10%
- Database query response: <100ms

10.4 Compliance and Standards

Security Standards:

- OWASP Top 10 compliance
- NIST Cybersecurity Framework alignment
- Local data privacy regulations

Development Standards:

- PEP 8 Python coding style
- IEEE 830-1998 SRS compliance
- Agile development methodologies

Document Information

Document Version: 1.0 **Last Updated:** Current Date **Prepared By:** Development Team **Approved By:** Project Manager **Classification:** Internal Use

This document serves as the authoritative specification for the CyberGuard Pro cybersecurity application development project.